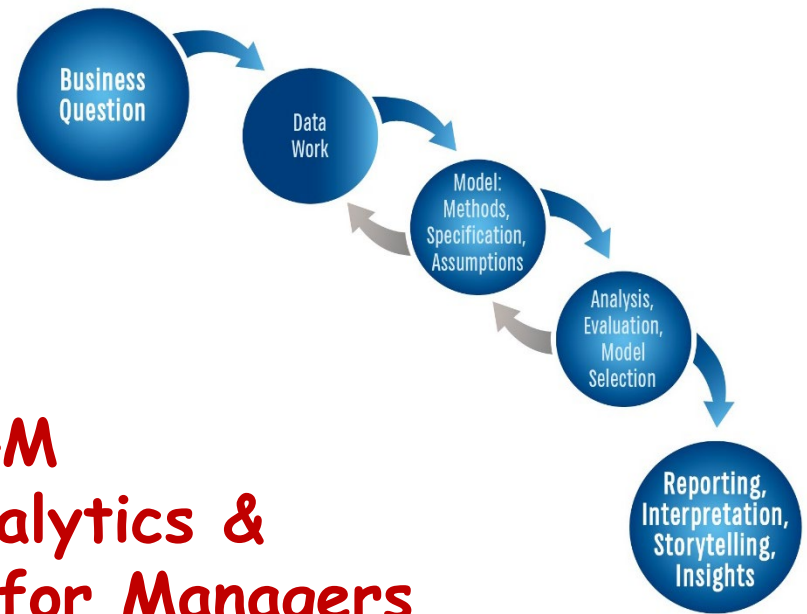




PAML4M
**Predictive Analytics &
Machine Learning for Managers**

4. Data Pre-Processing

PAML4M_4R_DataPreProcessing.Qmd

Prof. J. Alberto Espinosa
Last updated 8/1/2026



Overview



Variable Types

Variable Types

- Understanding variable types is the **first step** in pre-processing
- **Dependent** (predicted or response) vs. **Independent** (predictor) variables

 **Ex:** $\text{Income} \leftarrow f(\text{Age} + \text{Education Level} + \text{School} + \text{City})$ 

- **Variable Content Types:**

- **Numeric:**

- **Continuous** – e.g., 4.32, 0.48 – can do math
 - **Discrete** – e.g., 10, 20, 30, etc. – no decimals, can do math
 - **Binary** – 0, 1 (e.g., no/yes; approved/declined; sold/not)

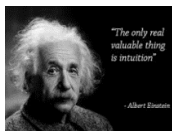
- **Character:**

- **Ordinal** – e.g., 1-7 survey scale (can't do math, e.g., SET's)
 - **Categorical** or **Factor** – e.g., Foreign/Domestic; Urban/Suburban/Rural
 - **Text** – e.g., E-mail subject or content, speech transcriptions

Why Pre-Process Data?

Introduction

- As we discussed earlier, two key steps in the business analytics cycle are:
(1) **Model Method Selection**; and (2) **Model Specification**.
- One important aspect of developing a **Model Specification** predictive modeling is **Feature Engineering**
- For any model tried or selected, **Feature Engineering** is about manipulating and preparing variables for the **Model Specification** step.
- Two key aspects of **Feature Engineering** include:
 1. **Data Pre-Processing and Transformations (this week)** – e.g., logs, lagging, categorical to binary, centering, standardizing, aggregating rows, etc.
 2. **Variable Selection (in 2 weeks)** – i.e., identifying the optimal set of predictors for the problem at hand.



Data Pre-Processing: Intuition

- About 80% to 90% of the work in analytics is getting the data ready for analysis.
- Data is often pre-processed to correct data problems
- But pre-processing is sometimes done to improve model performance – “feature engineering”
 - ✓ For example, a popular technique to detect spam mail is to look for the proportion of upper-case letters in the subject. This requires pre-processing the subject headers.
- Pre-processing needs are often based on the functional domain knowledge rather than mathematical reasons

Why Pre-Process Data?

- Typical **problems** with the data include: missing values; inconsistent data; outliers; skewed or bi-modal distributions.
- Sometimes the data is simply not in the **appropriate format** for the type of analysis to be done.
- Sometimes the data needed for the analysis needs to be gathered from **multiple data sources** and joined
- Or the data needs to be **scaled**, **centered** or **normalized**
- Or it needs to be **aggregated**, **summarized**, etc.
- Especially when data is not at the “**unit of analysis**” level
 - ✓ For example, you may have data on prices and features for individual houses, but you need to analyze housing by counties in the US – i.e., the unit of analysis is the **county**, not the individual homes → the data needs to be **aggregated**

Data Transformations

Predictors, Outcome or Both?

Key Modeling Data Issues: Independent (Predictor) Variables

Predictors:

- Can be continuous, discrete, binary, categorical or ordinal
- With large samples (50+ data points) predictors don't have to be normally distributed
- With small samples, sometimes they need be (for OLS, etc.)
- Categorical (Factor) variables need to be transformed into dummy variables. R does this for you.
- Categorical (Factor) variables can also be used to aggregate data by categories (e.g., counts, sums, average)
- Text data often needs to be pre-processed to create metadata (e.g., word counts, character counts, event counts, matching text, synonyms, etc.)

Key Modeling Data Issues: Dependent (Outcome) Variables

- For OLS, the outcome variable needs to be normally distributed
- **Note:** what really matters is that the regression residuals are normally distributed. But if the outcome variable is normally distributed, the residuals will generally be normally distributed too.
- The type of dependent (outcome or response) variable determines what type of modeling methods can be employed. If Y is →
 - ✓ Continuous → Regression and regression trees
 - ✓ Count (discrete, non-negative) → GLM with Poisson distribution
 - ✓ Binary → Classification models like, logistic regression, classification trees, discriminant analysis, etc..
 - ✓ Categorical (Factor) → Multinomial logistic regression, multinomial discriminant analysis, classification trees
 - ✓ Ordinal → Ordered Logistic, Ordered Probit regressions
 - ✓ Time Sensitive → Forecasting or time-series regression models



Data Transformation Overview

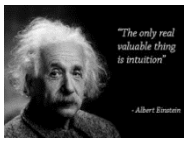
1. (Pred) Categorical to Dummy Variable → `relevel()`
2. (Pred) Polynomials → $I(x^2)$, $I(x^3)$, etc.
 - Later in the semester → `poly(x, n)` function → $x^1 + x^2 + \dots + x^n$
3. Log Transformations → `log()`
 - (Outc & Pred) Linear-Log, Log-Linear, Log-Log (Elasticity) Models
 - (Outc) Logit Models → `glm()` → `family = binomial(link = "logit")`
 - (Outc) Count Data → `glm()` → `family = poisson(link = "log")`
4. (Outc & Pred) Centering → `scale(..., center = T, scale = F)`
5. (Outc & Pred) Standardization → `scale(..., center = F, scale = T); lm.beta()`
6. (Outc & Pred) Lagging Data → `dwtest()`; `slide(){DataCombine}`

[\[Top\]](#) Transformation #1:

Predictors:

Categorical to Dummy Variables

OLS LI(*) - Relationship between Y and X's are not linear



Categorical Variables: Intuition

- Categorical (i.e., **factor**) variables are **not quantitative**, so the assumption of linearity does not hold. But a very simple **transformation** using **dummy variables** solves this problem.
- For example, if you have a categorical variable called **State**, it will contain a text value of 2 upper case letters with a value corresponding to one of the 50 U.S. states.
- If you want to predict housing prices, the **State** where the house is located will probably make a difference. So, how do we **model** the **effect** of **State** location?
- The answer is to **convert** the categorical variable **State** into 50 **dummy variables**, each named after the corresponding **State**, with a value of **1** if the house is in that **State** and **0** otherwise.
- For example, the variable **MD** would have a **1** for all houses in the state of Maryland and **0** for any of the other states.
- This will yield **50** dummy (**quantitative**) variables to model

The Dummy Variable Trap

- This is a well-known problem when you convert a categorical variable into various “mutually exclusive” dummy variables.
- For example, if you have a categorical variable called “LocationType” and it has one of three possible values (Urban, Suburban and Rural) we can create 3 dummy variables called Urban, Suburban and Rural, respectively.
- If `LocationType = “Urban”` → `Urban = 1; 0 otherwise`
If `LocationType = “Suburban”` → `Suburban = 1; 0 otherwise`
If `LocationType = “Rural”` → `Rural = 1; 0 otherwise`
- However, these three dummy variables are mutually exclusive, so if `Urban = Suburban = 0`, then `Rural` has to be 1, etc.
- So, the value in any of these variables is dependent on the other 2
- Including all 3 variables in a regression model will not only violate the assumption of independence, but will also create infinite multicollinearity and infinite standard errors

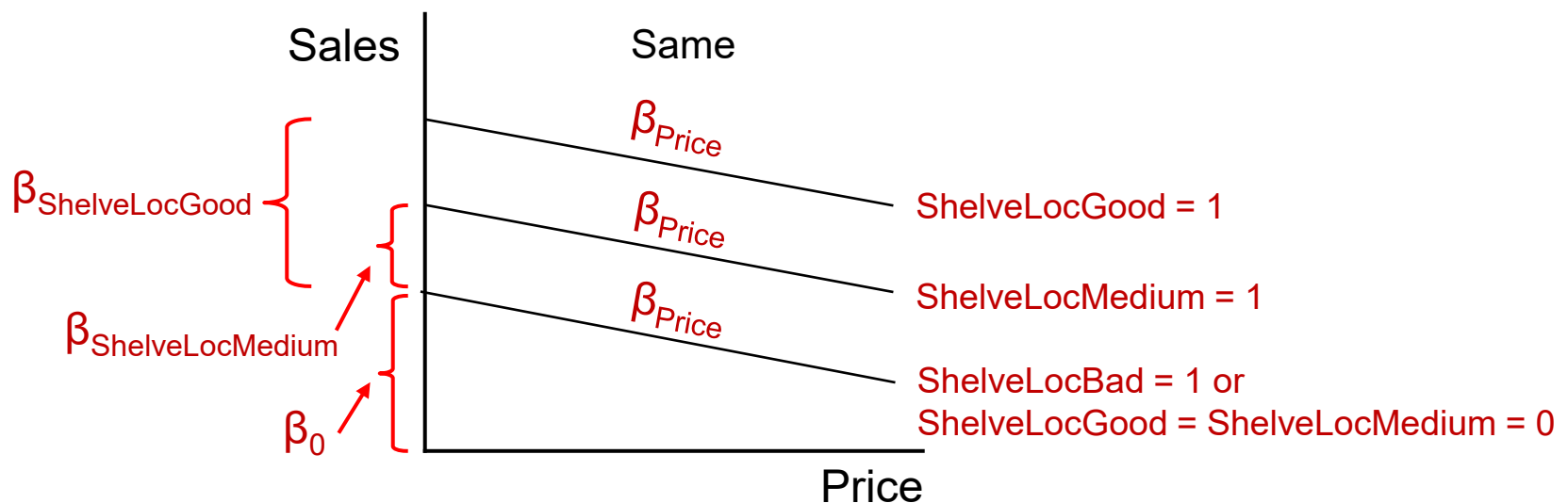
Modeling Categorical Predictors

- If an **independent variable** is **categorical**, you can convert it to **N dummy variables**, where **N** is the number of categories in the data for that variable → **R** does this **automatically** for you
 - Due to the **dummy variable trap**, you can only **model N-1** dummy variables.
 - The variable **left out** is called the “**baseline**” or “**reference**” variable (e.g., MD)
- The regression **intercept** represents the effect of the **reference variable**, when all other variables are 0 (e.g., house value for houses in MD – i.e., all **included** dummy variables = 0).
 - A **coefficient** for any of the other variables (e.g., VA) represents the **effect difference** between that variable (e.g., VA) and the reference variable (e.g., MD), all else equal

Categorical Predictor Illustration

- If you wish to predict car seat sales, using shelf location and price as predictors, and there are 3 possible shelf locations, R creates 3 dummy variables:
 - If `ShelveLoc = "Bad"` $\rightarrow$ `ShelveLocBad = 1`; 0 otherwise
 - If `ShelveLoc = "Medium"` $\rightarrow$ `ShelveLocMedium = 1`; 0 otherwise
 - If `ShelveLoc = "Good"` $\rightarrow$ `ShelveLocGood = 1`; 0 otherwise
- And **drops** the first one alphabetically (`ShelveLocBad`) unless you `relevel()`

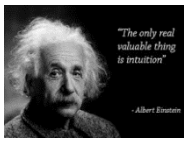
`lm(Sales ~ Price + ShelveLoc, data = Carseats)`



Categorical (Factor) Variables in R

Like most modern statistical software:

- If you **include** a **categorical variable** in a model, **R** will automatically convert it into **N - 1 dummy** variables
- By default, **R** will **leave out** the dummy variable corresponding to the first category **alphabetically** (to avoid the **dummy variable trap**)
- But often, there is a different categorical value that makes more sense to use as a reference level.
- In that case you can use the `relevel()` function to specify which dummy variable to **leave out** as a **reference**



Categorical to Dummy: Intuition

- As we discussed earlier, a **categorical** variable with **N** distinct categories can be transformed into **N dummy variables**
- For example, this model:
StudentRetention = f (GPA, **Major**, **Class**)
Has **1 continuous** and **2 categorical** variables, for example
Class can be **Freshman, Sophomore, Junior, Senior**
- We can create a dummy variable **Junior = 1** if the student is a Junior, **0** otherwise, etc.
- Because of the **dummy variable trap**, we can only model 3 of the 4 variables (i.e., N-1 variables)
- And leave one out as the reference level

E2	:			f_x	=IF(\$B2="Junior", 1,0)	
	A	B	C	D	E	F
1	Student	Class	Freshman	Sophomore	Junior	Senior
2	Espinosa	Junior	0	0	1	0
3	Lee	Sophomore	0	1	0	0
4	Carmel	Junior	0	0	1	0
5	Klein	Senior	0	0	0	1
6	Mortati	Freshman	1	0	0	0
7	Armour	Sophomore	0	1	0	0
8	Cakici	Junior	0	0	1	0
9	Karaesmen	Senior	0	0	0	1

Converting Categorical to Dummy in R

- Look at the categorical values you are interested in analyzing
- And create one binary variable for each category of interest
- R does this for you. It creates dummy variables named using the factor **variable name**, followed by the **category** they represent

Income	Loc
80,000	Urban
90,000	Urban
60,000	Suburban
100,000	Urban
45,000	Rural
75,000	Suburban
30,000	Rural

LocUrban	LocSuburban	LocRural
1	0	0
1	0	0
0	1	0
1	0	0
0	0	1
0	1	0
0	0	1

IMPORTANT !! When working with categorical variables you need to have a sufficient number of observations in each category. As a rule of thumb, it is recommended to merge categories with <10 observations.



Categorical Variables

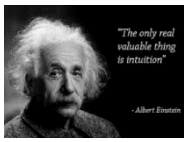
- {library} Topic
Search keywords
- {ISLR} Categorical to Dummy Variables Transformation
`class (Carseats$ShelveLoc)`
`Carseats$ShelveLoc <- relevel (Carseats$ShelveLoc,`

[\[Top\]](#) Transformation #2:

Predictors:

Polynomials

OLS LI(x) - Relationship between Y and X 's are not linear



Polynomials: Intuition

- More on this later on the **Non-Linear Models** lecture
- Polynomial transformations are very useful when the relationship between the **X's** and **Y** are suspected to be **non-linear**
- When **scatter plots** suggest a non-linear relationship between **x** and **y**, a quadratic model $y = \beta_0 + \beta_1x + \beta_2x^2 + \varepsilon$ or higher polynomial transformations (with β_3x^3 , β_4x^4 , etc.) are preferred.
- A **quadratic** regression curve has at most **1 peak/valley**, a **cubic** has at most **2**, etc.
- The **higher** the **polynomial** the **wavier** the regression curve
- The problem is that **high degree polynomials** are **difficult to interpret** and can “**over-identify**” the model and do not generally perform well with new data, especially at **both ends** of the curve
- **Spline** and **piecewise** models (**covered later** in the class) generally perform better than high polynomials.

Polynomials

- {library} Topic
Search keywords
- Linear Model

```
lm.fit1 <- lm(medv ~ lstat, data = Boston)
```
- Quadratic Model

```
lm.fit2 <- lm(medv ~ lstat + I(lstat^2))
```
- Cubic Model

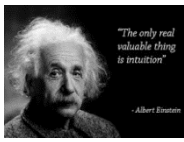
```
lm.fit3 <- lm(medv ~ lstat + I(lstat^2) + I(lstat^3),
```
- ANOVA

```
anova(lm.fit1, lm.fit2, lm.fit3)
```
- Polynomial 5th Degree

```
lm.fit5 <- lm(medv ~ poly(lstat, 5, raw = T),
```

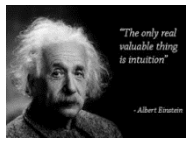

[\[Top\]](#) **Transformation #3:**
Predictors & Outcome:
Log Models

OLS LI(x) - Relationship between Y and X 's not linear, or
OLS YN(x) - Y is not normally distributed



Importance of Log Models

- You can **log** the **response** variable **Y** or any of the quantitative **predictors X's** that can **never** be **negative** (i.e., salaries, enrollments, miles per gallon, etc.)
- If the response variable **Y** is **not normal** (OLS assumption), logging may (or may not) solve the problem, but no need to do this when the **residuals** are normally distributed
- Both, the response **Y** and some predictors **X** are sometimes logged for **convenience** (e.g., to improve interpretation, statistical power, elasticity models).
- But → logging **changes** the **interpretation** of the coefficients.



Log Models: Intuition

- Log transformations are useful when the distribution of the data is **skewed** to the right → the distribution usually becomes more normal
- You can inspect a variable for **normality visually** with:
 - **QQ-Plots** – shows data departures from perfect normality.
 - **Histograms** – curve should be bell-shaped
- You can also test for normality **quantitatively** with tests like **Shapiro-Wilks**, but there are 2 caveats:
 - This test is useful with small samples, but it is notorious for **rejecting normality** with large samples → always inspect visually
 - For this reason, the `shapiro.test(){stats}` function does not allow samples larger than **5,000** (you can take a smaller random sample)
- **Cannot log** variables with **negative values** (no logs for negatives)
- Useful with **truncated data** (e.g., count data – truncated at 0)
- Useful to model **elasticity (%) effects** (explained shortly)



Log Models

- `{library}` Topic
Search keywords
- `{ggplot2}` Checking Normality
`hist(diamonds$price,`
`shapiro.test(diamonds[sample(nrow(diamonds), 5000), ]$price)`
- Correcting for Non-Normality with Logs
`hist(log(diamonds$price),`

Log Models: Interpreting Effects

A model $y = \beta_0 + \beta_1 x + \varepsilon$ has 4 possible linear-log models
(more than one x variable can be logged):

Outcome Variable	Predictor Variable	
	x	$\text{Log}(x)$
y	Linear Model $y = \beta_0 + \beta_1 x + \varepsilon$ $x \uparrow 1 \text{ unit} \rightarrow y \uparrow \beta_1 \text{ units}$	Linear-Log Model $y = \beta_0 + \beta_1 \text{Log}(x) + \varepsilon$ 1% (1/100) $\uparrow$ in $x \rightarrow y \uparrow \beta_1/100 \text{ units}$
$\text{Log}(y)$	Log-Linear Model $\text{Log}(y) = \beta_0 + \beta_1 X + \varepsilon$ $x \uparrow 1 \text{ unit} \rightarrow y \uparrow 100 * \beta_1 \% \text{ (i.e., } \beta_1 \text{ fraction)}$	Log-Log (Elasticity) Model $\text{Log}(y) = \beta_0 + \beta_1 \text{Log}(x) + \varepsilon$ 1% $\uparrow x \rightarrow y \uparrow \beta_1 \%$



Log Transformations

- {library} Topic

Search keywords

- Possible Log Transformations

```
summary(lm(price ~ carat, data = diamonds))
```

```
summary(lm(price ~ log(carat), data = diamonds))
```

```
summary(lm(log(price) ~ carat, data = diamonds))
```

```
summary(lm(log(price) ~ log(carat), data = diamonds))
```

- Correcting for Non-Normality with Logs

```
hist(log(diamonds$price),
```

More on Log-Log (Elasticity) Models

- In some models (popular in **economics**) the predictor effects are **not linear** but **multiplicative**
- One interesting aspect of multiplicative effects is that if just **one of the predictors is 0, the outcome is 0**. In contrast, if one predictor is 0 in a linear model but there are other predictors, the outcome is not 0
- Multiplicative elasticity model specification:

$$y = c x_1^{\beta_1} x_2^{\beta_2} x_3^{\beta_3} \rightarrow \text{now, take logs on both sides}$$

$$\begin{aligned} \text{Log}(y) &= \text{Log}\left(c x_1^{\beta_1} x_2^{\beta_2} x_3^{\beta_3}\right) \\ &= \text{Log}(c) + \text{Log}\left(x_1^{\beta_1}\right) + \text{Log}\left(x_2^{\beta_2}\right) + \text{Log}\left(x_3^{\beta_3}\right) \\ &= \beta_0 + \beta_1 \text{Log}(x_1) + \beta_2 \text{Log}(x_2) + \beta_3 \text{Log}(x_3) \end{aligned}$$

- Elasticity (log-log) models have a simple, intuitive explanation, which is useful for certain economic models:
 $\rightarrow$ when x_1 goes up by **1%**, y changes up by $\beta_1\%$



[\[Top\]](#) Transformation #3.1:

Outcome:

Logit-Transformed Models

(already covered in the Regression lecture)

See: # Logit-Transformed Models (Binary Y) in script

OLS LI(x) - Relationship between Y and X's not linear,

OLS YN(x) - Y is not normally distributed, or

OLS YC (x) - Y is not continuous $\rightarrow$

binary: **0,1**; yes/no; success/failure, etc.

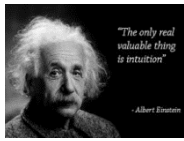
[\[Top\]](#) Transformation #3.2:

Outcome:

Log-Transformed Models for Count Data

OLS $YC(x)$ - Y is not continuous, but integer counts,
truncated at 0

OLS $EV(x)$ - The error variance is not constant

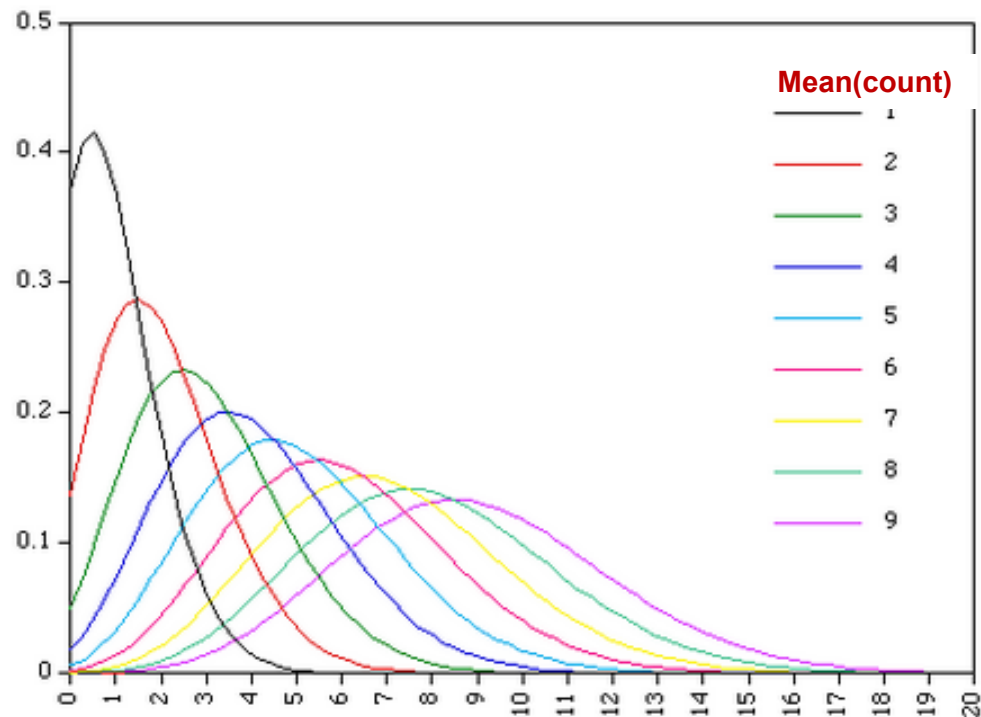


Count Data Models: Intuition

- Predicting counts is a **common problem** in analytics – e.g., number of students enrolled, number of customers in a store, number of votes in an election, number of days to sell a house
- It is **not uncommon** to see predictive models with counts as an outcome variable using **OLS**
- But this is **incorrect** for a number of reasons, including
 - Counts are **discrete**, not continuous (i.e., no decimals)
 - Counts are **positive** – i.e., can't be less than 0 (i.e., data is **"truncated"** at 0.
 - The **distribution** of count data is **not normal**
 - The **error variance** is **not constant** – relatively low near 0 and increasing as counts get larger
 - In many count data sets, there is a **disproportionate** amount of **0's**

The Poisson Distribution

- The **Normal** distribution: is **symmetrical** around the mean; it's **tails** extend **indefinitely** at both ends; and it is **continuous**.
- The **Poisson** distribution: is bounded at **0**; is **asymmetrical**; **varies** in **shape** as the mean increases (it approaches a normal distribution when counts have very wide ranges)
- As it turns out, **count data** follow a **Poisson** distribution
- Notice in the diagram how the **shape** of the distribution curve **changes** with the **count mean** and how it becomes **more normal** with **large** means.



Count Data Models: Details

- **Log-transforming** the count data (i.e., outcome variable) and assuming a **Poisson distribution** for the counts yields more suitable regression models to predict count data:

$$\text{Log}(y) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \varepsilon$$

- This type of regression is called “**Poisson**” model
- The **interpretation** of the coefficients is the same as the **Log-Linear** model explained before:

$$x \uparrow 1 \text{ unit} \rightarrow y \uparrow 100 \cdot \beta_1 \% \text{ (or } \beta_1 \text{ as a fraction)}$$

- This model can be estimated with **OLS**
- But it is customary to estimate it using a “**Generalized Linear Model**” (GLM) regression using **Maximum Likelihood Estimation** (MLE), which we explain later in the chapter.
- It sounds complicated, but as we will explain later (see the binary logistic lecture), it can be fit in **R** using the **glm()** function by specifying the attribute **family = poisson(link = "log")**

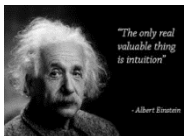


Count Data Models

- {library} Topic
Search keywords
- {ISLR} Count Data Models
`glm.fit.count <- glm(SalesUnits ~`

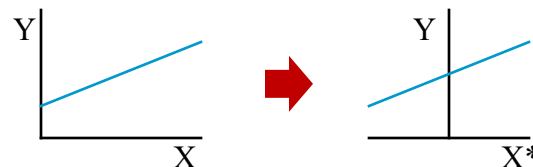
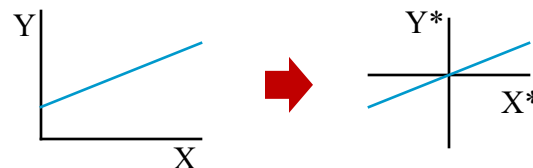
[\[Top\]](#) **Transformations #4: Centering**
Outcome and Predictors:

Same as OLS



Centering: Intuition

- Centering is generally useful when the **intercept is meaningless** because **x cannot be 0** (e.g., predicting cholesterol based on weight → nobody has 0 weight). Centering the variable shows the effect of x when x is at its mean value (not zero).
- Centering** x_1 , x_2 and y with respect to their means helps →
$$x_1^* = x_1 - \bar{x}_1; x_2^* = x_2 - \bar{x}_2$$
- This is **equivalent** to **shifting** the y and x **axes** to their respective means
- If you only center some x 's, then, the intercept is the effect when these x 's are at their means.
- Only the intercept changes → β coefficients, R^2 and p -values **do not change**





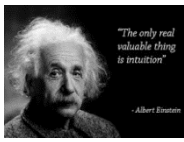
Centering Data

- {library} Topic
Search keywords
- {MASS} Centering Data
`Boston.centered <- data.frame(scale(Boston,`

[\[Top\]](#) Transformations #5: Standardization

Outcome and Predictors:

Same as OLS



Standardization: Intuition

- If you **divide** a **centered** variable by its **standard deviation** the result is a “**standardized**” variable or “**z-score**”:

$$y^* = \frac{y - \bar{y}}{\sigma_y} \quad x^* = \frac{x - \bar{x}}{\sigma_x}$$

- Such that $\text{mean}(y^*) = \text{mean}(x^*) = 0$; and $\sigma(y^*) = \sigma(x^*) = 1$
- This is useful when **comparing dissimilar scales** (e.g., is the weight effect larger than engine size effect?); and when the **effect size** of an unstandardized variable has **no meaning** (e.g., survey 1-7 scale responses).
- Also, some modeling **methods standardize** variables (e.g., Ridge, LASSO) automatically, so it is important to understand what it means

Effects:

x^* **increases by** 1 standard deviation →

y^* **changes** β_1 standard deviations



Standardizing Data

- {library} Topic
Search keywords
- {MASS} Standardizing Data
`Boston.standardized <- data.frame(scale(Boston,`

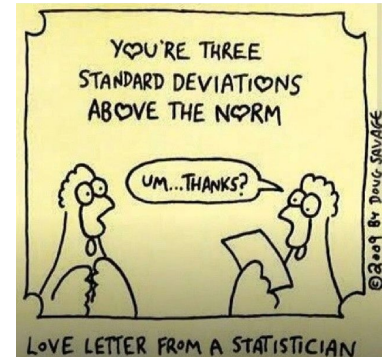
Standardized Regression

- **Fun Fact 1:** in a **simple regression** model with **one** predictor, the resulting **standardized coefficient** is identical to the **correlation** between **y** and **x**.
- **Fun Fact 2:** if you **standardize** all **variables** or if you standardize the **raw β coefficients** you get the exact **same results**. That is, if you fit an **unstandardized** regression model, you can extract standardized **β coefficients** (**R** does this for you):

$$\beta_x^* = \beta_x \frac{\sigma_x}{\sigma_y}$$

- The **p-values** and **R^2** of a **standardized** regression are **identical** to those of a regular regression, but the coefficients' **value** and **interpretation** change.

You can run an unstandardized model and then extract standardized coefficients with the `lm.beta()` function in a single model.





Standardized Regression

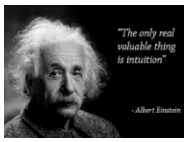
- {library} Topic
Search keywords
- {lm.beta} Extracting Standardized Coefficients
`lm.standardized <- lm.beta(lm.fit)`

[\[Top\]](#) **Transformation #6:**
Predictors:
Lagging Data

EI(x) - Errors are not independent

Forecasting Models

- Models aimed at making predictions **outside the existing data range** are called **forecasting models**. Like any forecasting (e.g., the economy, the weather), the **prediction confidence** declines as you get farther away from the existing data range
- The difficulty with forecasting, is that **present values of the dependent variable** (e.g., your height today) are sometimes highly **correlated** with **prior values** (e.g., your height yesterday), which violates a key OLS assumption of **data independence** or **error independence**
- This problem is known as **serial correlation** or **auto correlation**
- **Lagging** can help **reduce serial correlation**, although there are more sophisticated and robust forecasting models (outside our scope)
- Most lagged models predict over time → **Time Series Models**



Time Series Models: Intuition

Time series models come in different flavors of **three main types**

- **(Univariate) Time Series** – predicting future values of a variable based on its prior historical values (e.g., predict tomorrow's weather)
 - ✓ Forecasting methods → moving averages, exponential smoothing, etc.
- **Multivariate Time Series** – like time series, but also include other **predictors** and **control** variables
 - ✓ Predictive models → Check for dependencies between observations, like serial correlation, time effects, etc.
- In depth analysis of forecasting and panel data models is **beyond the scope** of this course, but **time effects** need to be tested and **lagging** outcome or predictors may overcome some issues.

Multivariate Time Series Models

- Say, we want to forecast monthly **house starts** (in thousands; **KUnits**)
- Because **KUnits** in one month may be highly correlated with **KUnits** in prior months → **serial correlation** is a concern
- Lagging the **KUnits** (the outcome variable) may help correct for this → **Univariate Time Series**
- The table below shows **KUnits** lagged by 1 to 4 months
- Based on **business knowledge** you may want to **include** additional variables **model** (e.g., S&P, quarters) → **Multivariate Time Series**

Month	T	KUnits	KUnits_L1	KUnits_L2	KUnits_L3	KUnits_L4	S&P	Q1	Q2	Q3	Q4
Jan 1990	1	99.2					329.08	1	0	0	0
Feb 1990	2	86.9	99.2				331.89	1	0	0	0
Mar 1990	3	108.5	86.9	99.2			339.94	1	0	0	0
Apr 1990	4	119.0	108.5	86.9	99.2		330.80	0	1	0	0
May 1990	5	121.1	119.0	108.5	86.9	99.2	361.23	0	1	0	0
Jun 1990	6	117.8	121.1	119.0	108.5	86.9	358.02	0	1	0	0
Jul 1990	7	111.2	117.8	121.1	119.0	108.5	356.15	0	0	1	0
Aug 1990	8	102.8	111.2	117.8	121.1	119.0	322.56	0	0	1	0

Multivariate Time Series & Lags

economics dataset in `{ggplot2}` package

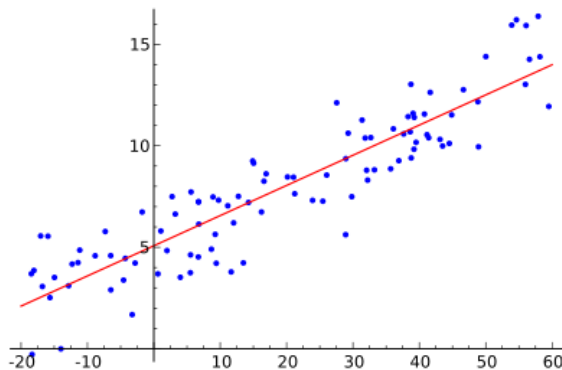
- `pce` → personal consumption expenditures (\$ billions)
- `pop` → total population (thousands)
- `psavert` → personal savings rate
- `uempmed` → median duration of unemployment in weeks
- `unemploy` → number of people unemployed (thousands)

date	pce	pop	psavert	uempmed	unemploy	unemploy.L1	unemploy.L12
7/1/1967	506.7	198,712	12.6	4.5	2,944		
8/1/1967	509.8	198,911	12.6	4.7	2,945	2,944	
9/1/1967	515.6	199,113	11.9	4.6	2,958	2,945	
10/1/1967	512.2	199,311	12.9	4.9	3,143	2,958	
11/1/1967	517.4	199,498	12.8	4.7	3,066	3,143	
12/1/1967	525.1	199,657	11.8	4.8	3,018	3,066	
1/1/1968	530.9	199,808	11.7	5.1	2,878	3,018	
2/1/1968	533.6	199,920	12.3	4.5	3,001	2,878	
3/1/1968	544.3	200,056	11.7	4.1	2,877	3,001	
4/1/1968	544.0	200,208	12.3	4.6	2,709	2,877	
5/1/1968	549.8	200,361	12.0	4.4	2,740	2,709	
6/1/1968	556.3	200,536	11.7	4.4	2,938	2,740	
7/1/1968	563.2	200,706	10.7	4.5	2,883	2,938	2,944
8/1/1968	567.0	200,898	10.5	4.2	2,768	2,883	2,945
9/1/1968	568.2	201,095	10.6	4.6	2,686	2,768	2,958
10/1/1968	571.6	201,200	10.8	4.8	2,600	2,686	2,743

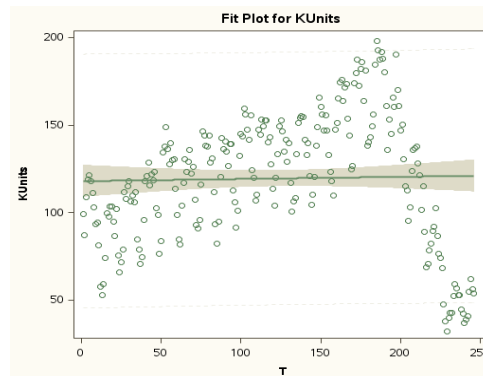
Serial Correlation

Serial Correlation

- One important **OLS regression assumption** is that the errors between various observations are **not systematically correlated**.
- When errors are correlated, this provides an indication that we are **missing something** in our model that makes the errors exhibit **systematic** patterns (errors should look like a “cloud”).
- This can be observed visually by plotting the **sequence variable** (e.g., **time**) against the **predicted variable**.
- It can also be tested with a **Durbin-Watson, Box-Pierce, Ljung-Box** or similar test

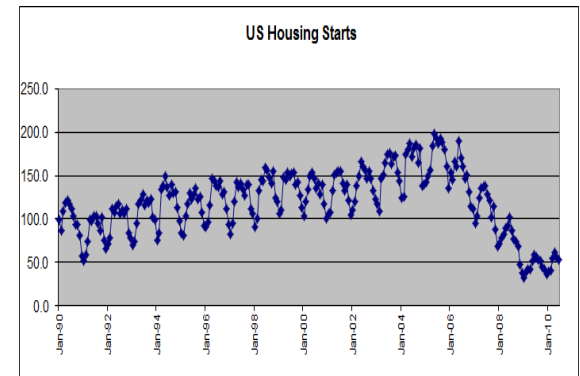


No Serial Correlation



Curvilinear

Serial Correlation



Cyclical



Visual Inspection for Serial Correlation

- `{library}` Topic
Search keywords
- `{ggplot2}` Visual Inspection for Serial Correlation
`plot(economics$date, fit.no.lag$residuals,`

Testing for Serial Correlation

- If **serial correlation** is present means that if you order your data by the ordinal variable (e.g., by time), then an **error at time t** will be highly **correlated** with the error at time $t-1$ and $t+1$ – i.e., errors next to each other are very similar.
- If the data is **seasonal** (e.g., sales fluctuate by quarter or by month of the year), the data maybe strongly correlated by season – e.g., errors in **Q1** is highly correlated with errors in **Q1** the **prior year**; or errors in December are highly correlated with errors in December the prior year.
- There are various tests for serial correlation → they all focus on evaluating the **correlation** or **difference** between **errors** of **nearby observations**. The most popular are:
 - ✓ **Durbin-Watson** (squared difference of errors, 1-lag apart)
 - ✓ **Breusch-Godfrey** (regress errors on prior errors and check the R^2)
 - ✓ **Box-Pierce** and **Ljung-Box** (compares errors n-lags apart)

Durbin-Watson (DW) Test

- The **DW** statistic is defined mathematically as: $\frac{\sum (e_i - e_{i-1})^2}{\sum e_i^2}$
- Squaring is done to convert negative differences into positives
- It basically means **how much contiguous residuals deviate**, relative to the errors themselves (to take care of scaling issues)
- The **DW** statistic falls between 0 and 4
 - ✓ **DW = 0** → Extreme **positive** serial correlation
 - ✓ **DW = 2** → **No** serial correlation
 - ✓ **DW = 4** → Extreme **negative** serial correlation
 - ✓ **1.5 < DW < 2.5** ($p \geq 0.05$) → **Generally OLS regression is OK**
 - ✓ Otherwise ($p < 0.05$) → Need a forecasting model that corrects serial correlation
 - ✓ **Lagging variables** is a simple method that sometimes corrects serial correlation



Durbin - Watson test for Serial Correlation

- `{library}` Topic
Search keywords
- `{ggplot2}` Visual Inspection for Serial Correlation
`dwtest(fit.no.lag)`

What and How Much to Lag?

- Serial correlation may not be with residual in consecutive data points, but perhaps in data **n periods apart** (e.g., 3-month quarter, 12-month year, etc.).
- We can test serial correlation with residuals n-periods apart with tests like **Box-Pierce** and **Ljung-Box**.
- **N-period lagging** can help **reduce n-period serial correlation** and you can lag outcome, predictor or both variables.
- For example, to predict **income next year**, your **income last year** and your **education 10 years ago** may be good predictors:

$$\text{Income}_t = \beta_0 + \beta_1 \text{Income}_{t-1} + \beta_2 \text{Education}_{t-10} + \text{etc.}$$

- Or perhaps you are trying to predict **quarterly sales** this year based on several factors and also based on the sales volume during the **same quarter** in the prior year (**t** in months):

$$\text{QtrlySales}_t = \beta_0 + \beta_1 \text{QtrlySales}_{t-4} + \beta_2 \text{MktgExpense} + \text{etc.}$$



Test for Multi-Period Serial Correlation

- {library} Topic
Search keywords
- {stats} Box-Pierce and Ljung-Box Tests for Serial Correlation
`Box.test(residuals(fit.no.lag), lag = 12)`

Building Multivariate Time Series Models & Correcting for Serial Correlation

- Specify a **multivariate model** without lagged variables
- Conduct a **DW, BG, BP or LB tests** for serial correlation
- If $1.5 < DW < 2.5$ or if **BG** are not-significant → model is **OK**
- If the test you chose yields a significant p-value
- **Lag** response and/or predictor variables as needed based on **business knowledge**
- **Check again** for **serial correlation**; repeat until you get an acceptable model



Correcting for Serial Correlation

- {library} Topic
Search keywords
- {DataCombine} Lagging Data
`economics <- slide(economics,`
- Correcting for Serial Correlation
`fit.lag <- lm(unemploy ~ date + unemploy.L1 + unemploy.L12 +`

Other Transformations

Other Transformations

- There are **endless options** for data transformations in pre-processing. We have covered the most popular ones.
- Examples of other transformations:
 - **Box-Cox** (see goodies)
 - **Ranks** (see goodies)
 - **Re-scaling**: e.g., from $^{\circ}\text{F}$ to $^{\circ}\text{C}$, mpg to kpg
 - **Reverse scaling**: often used to facilitate interpretation – e.g., a 1-7 satisfaction scale can be converted into a dissatisfaction scale by subtracting the value from 8, so that a 1 becomes 7 and a 7 becomes 1
 - **Inverse**: $x^* = \frac{1}{x}$ similar purpose than revers scaling, but this is a non-linear transformation, harder to interpret, and x cannot be